[Webinar] How Talkdesk Runs Enterprise-Grade OpenTofu with Spacelift ➡️ Register Now

TERRAFORM

9 Terraform Use Cases for Your Infrastructure as Code

Flavius Dinu
31 May 2024 · 7 min read

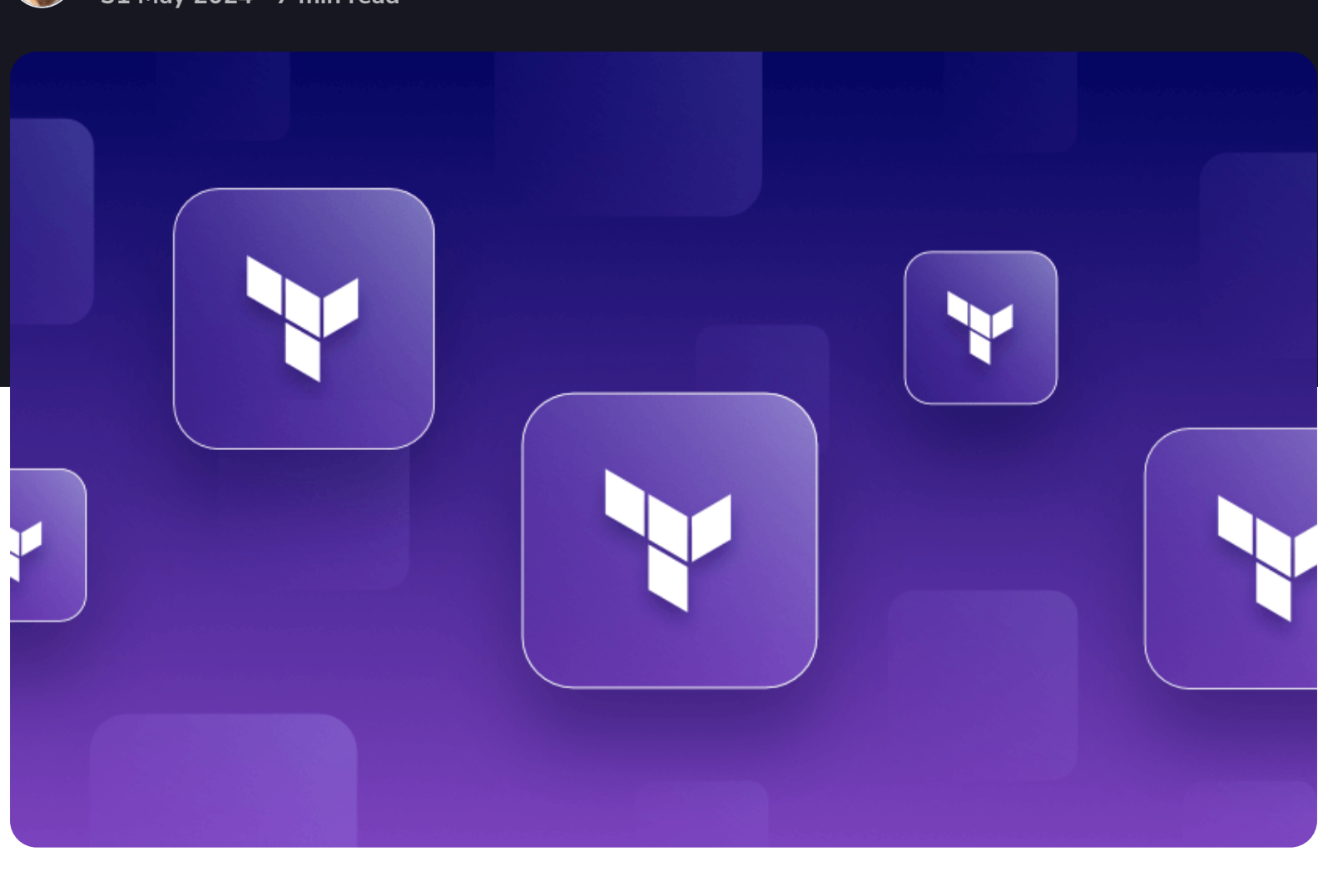

TABLE OF CONTENTS
What is Terraform?
Terraform use cases
Key points

Subscribe to our Newsletter

Mission Infrastructure newsletter is a monthly digest of the latest posts from our blog, curated to give you the insights you need to advance your infrastructure game.

Your email address

Subscribe

Terraform is a powerful infrastructure as code tool that simplifies the provisioning and management of infrastructure as code resources. In this article, we will explore some of the most popular Terraform use cases, showcasing its capabilities for single-cloud and multi-cloud deployments, managing K8s deployments, implementing high availability and disaster recovery configurations, showing how to build consistent environments, and how to use it with infrastructure management tools.

What is Terraform?

Terraform is one of the most used infrastructure as code (IaC) tools. It allows IT professionals to define their infrastructure as code and automates the deployment and management of infrastructure across multiple cloud providers and services. If you are looking for a reliable and efficient way to manage your infrastructure's lifecycle, from provisioning and configuration to resource management, Terraform is a viable choice.

Terraform was initially open-source and has switched to a business source license (BSL) last year. If you are interested in an open-source tool, that was created as a fork from the last version of the open-source version, you can explore OpenTofu.

Terraform use cases

What are Terraform's use cases?

1. Single cloud deployment
2. Multi-cloud deployment
3. Handling Kubernetes deployments
4. Implementing high-availability and disaster-recovery configurations
5. Consistent environments
6. Generic CI/CD pipelines
7. Policy as code
8. Multi-infrastructure tools integrations
9. Infrastructure management integration

1. Single cloud deployment

With Terraform, it is very easy to deploy resources inside your cloud provider. You just need to navigate to the documentation related to your cloud provider and declare a provider configuration for it to handle authentication. Then, you can easily create your resources.

Multiple modules are available to get you started. You can browse them in the registry if you don't want to write much code yourself.

Here is a very basic example that handles the creation of a VPC in AWS:

```
provider "aws" {
  region = "eu-west-1"
}

resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
}
```

2. Multicloud deployment

You can use Terraform to make multi-cloud deployments in the same state file. You simply repeat the same steps for the second cloud provider. By using multicloud, you avoid vendor lock-in, enhance redundancy, and optimize costs.

Here is an example that handles the creation of a VPC in AWS and a virtual network in Azure:

```
provider "aws" {
  region = "eu-west-1"
}

resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "example" {
  name     = "example-resources"
  location = "West Europe"
}

resource "azurerm_virtual_network" "example" {
  name                = "example-network"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name
  address_space       = ["10.0.0.0/16"]
}
```

3. Handling Kubernetes deployments

Terraform has dedicated providers for both Kubernetes and Helm. This means you can spawn a Kubernetes cluster with Terraform, and you also have various ways to configure resources on that cluster.

Let's take a look at an example that spawns an EKS cluster and then install ArgoCD in it:

```
resource "aws_eks_cluster" "main" {
  name     = "main-eks-cluster"
  role_arn = "role-arn"
  vpc_config {
    subnet_ids = ["subnet1", "subnet2"]
  }

  tags = {
    Name = "main-eks-cluster"
  }
}

data "aws_eks_cluster_auth" "main" {
  name = aws_eks_cluster.main.name
}

provider "helm" {
  kubernetes {
    host                   = aws_eks_cluster.main.endpoint
    token                  = data.aws_eks_cluster_auth.main.token
    cluster_ca_certificate = base64decode(aws_eks_cluster.main.certificate_authority[0].data)
  }
}

resource "helm_release" "argocd" {
  name       = "argocd"
  repository = "https://argoproj.github.io/argo-helm"
  chart      = "argo-cd"
  version    = "4.5.2"

  namespace = "argocd"

  create_namespace = true

  set {
    name  = "server.service.type"
    value = "LoadBalancer"
  }

  set {
    name  = "server.service.annotations.service\\.beta\\.kubernetes\\.io/aws-load-balancer-type"
    value = "nlb"
  }
}
```

In the above example, our second provider (Helm), gets configured with information from the EKS cluster, ensuring it deploys the ArgoCD cluster inside it once it is created.

Read also: Terraform vs. Helm

4. Implementing high-availability (HA) and disaster-recovery (DR) configurations

Terraform facilitates the creation of HA/DR architectures by enabling the creation of resources in different availability domains and different regions. This ensures that services remain up and running even when there is an availability domain or a region failure.

Here is a DR example that shows how to create instances in different regions:

```
provider "aws" {
  alias  = "primary"
  region = "us-west-2"
}

provider "aws" {
  alias  = "dr"
  region = "us-east-1"
}

resource "aws_instance" "primary" {
  provider      = aws.primary
  ami           = "ami_id"
  instance_type = "t2.micro"

  tags = {
    Name = "primary-instance"
  }
}

resource "aws_instance" "dr" {
  provider      = aws.dr
  ami           = "ami_id"
  instance_type = "t2.micro"

  tags = {
    Name = "dr-instance"
  }
}
```

💡 **You might also like:**

- How to Manage Terraform at Scale
- Automating Infrastructure Deployments Using Terraform
- How to Manage Multiple Terraform Environments Efficiently

5. Consistent environments

Leveraging Terraform modules makes it very easy to replicate your configuration across multiple environments and build an input-driven solution.

I have built a very simple module that provisions one EC2 instance and has configurable parameters for the `ami_id` and `instance_type`:

```
resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
}

variable "ami_id" {
  type    = string
  default = "ami"
}

variable "instance_type" {
  type    = string
  default = "t2.micro"
}
```

Now, I can leverage this module in different configurations:

- **Dev environment:**

```
# dev/main.tf
module "instance_dev" {
  source = "../"
}
```

- **Stage environment:**

```
# stage/main.tf
module "instance_stage" {
  source        = "../"
  instance_type = "t3.micro"
}
```

- **Prod environment**

```
# prod/main.tf
module "instance_prod" {
  source        = "../"
  instance_type = "t3.micro"
}
```

Although the instance types differ from one environment to the other, we ensure consistency by using the same type of code and the same instance images for all of them.

6. Generic CI/CD pipelines

Terraform integrates with many tools to create a better workflow. Natively, its format and validate commands help you do the continuous integration part. With these options, you can check if the code respects the linting standards and is valid.

In addition, for the CI part, you can integrate with security vulnerability scanning tools such as tfscan, checkov, or terrascan to check if your code has any vulnerabilities before promoting it to the main branch.

Example GitHub actions, CI pipeline that checks format, validate, uses tfscan for vulnerability scanning, and runs a `terraform plan`:

```
name: Terraform CI Pipeline

on:
 push:
   branches:
     - main
 pull_request:
   branches:
     - main

jobs:
 terraform:
   name: Terraform Checks
   runs-on: ubuntu-latest

   steps:
   - name: Checkout code
     uses: actions/checkout@v2

   - name: Setup Terraform with specified version on the runner
     uses: hashicorp/setup-terraform@v3
     with:
        terraform_version: 1.5.7
        terraform_wrapper: false

   - name: Terraform Format Check
     run: terraform fmt -check

   - name: Terraform Validate
     run: terraform validate

   - name: Install tfscan
     run: |
       curl -LO https://github.com/Tfsec/tfsec/releases/latest/download/tfsec-linux-amd64
       chmod +x tfsec-linux-amd64
       sudo mv tfsec-linux-amd64 /usr/local/bin/tfsec

   - name: Run tfscan
     run: tfsec

   - name: Terraform plan
     run: terraform plan -no-color -input=false
```

The pipeline can be accommodated to do the CD part as well. In my example, I will run apply only when there is a merge to the main branch:

```
- name: Terraform Apply
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: terraform apply -auto-approve -input=false
```

7. Policy as code

Policy as code should go hand in hand with CI/CD pipelines. You can use OPA (Rego) or Sentinel to define policies and run them before doing apply. Policies help with governance and compliance so you can restrict resources or resource parameters to ensure your organization's requirements are respected.

Let's create a Rego policy that restricts all instances that differ from t3.micro:

```
package terraform

deny[msg] {
   resource := input.resource_changes[_]
   resource.type == "aws_instance"
   instance := resource.change.after
   instance.instance_type != "t3.micro"
   msg = sprintf("EC2 instance %s has an invalid instance type: %s", [resource.address, instance
}
```

Now, we will need to evaluate this against an input, so we will need to build a `terraform plan` in a JSON format. I will use the above example with the primary and dr instances.

```
terraform plan -out=tfplan.binary
terraform show -json tfplan.binary > tfplan.json
```

Now, let's run the OPA policy against our plan and see what happens:

```
opa eval -i tfplan.json -d restrict_t2_micro.rego "data.terraform.deny"
{
 "result": [
   {
     "expressions": [
       {
         "value": [
           "EC2 instance aws_instance.dr has an invalid instance type: t2.micro",
           "EC2 instance aws_instance.primary has an invalid instance type: t2.micro"
         ],
         "text": "data.terraform.deny",
         "location": {
           "row": 1,
           "col": 1
         }
       }
     ]
   }
 ]
}
```

Our instances have t2.micro types, so you can see they have an invalid instance type because we only allow t3.micro instances.

To learn more, check out How to Use Open Policy Agent (OPA) with Terraform.

8. Multi-infrastructure tools integrations

Terraform can easily integrate with other tools to build a seamless workflow. Integrate Terraform with Ansible or Kubernetes to deploy your infrastructure and your application in one go.

You can see examples of this in the following posts:

- Using Terraform and Ansible together
- Using Terraform and Kubernetes together

9. Infrastructure management integration

Terraform can be leveraged easily with an infrastructure management platform such as Spacelift to affect all the use cases we've discussed.

With Spacelift, you can easily:

- Build a CI/CD workflow for Terraform.
- Combine your Terraform code with Ansible/K8s/CloudFormation/Pulumi and send outputs from one to another.
- Use policies to restrict resource types, resource parameters, define how many approvals you need for applies, establish where to send notifications, and make decisions on what happens when a PR is opened or merged.
- Use dynamic credentials for your cloud providers.
- Integrate with any third-party tools.
- Build self-service infrastructure.

Key points

Terraform is a versatile tool that can be leveraged to build your infrastructure. Combining it with a product such as Spacelift unlocks all the use cases presented in this article and implements them without the hassle of defining all the steps you would need with a generic CI/CD pipeline.

If you want to learn more about Spacelift, create a free account today, or book a demo with one of our engineers.

Automate Terraform deployments with Spacelift

Automate your infrastructure provisioning and build more complex workflows based on Terraform using policy as code, programmatic configuration, context sharing, drift detection, resource visualization, and many more.

Learn more

Written by

Flavius Dinu

Flavius is a passionate Developer Advocate with an Infrastructure as Code mindset and expertise in DevOps & Cloud Engineering. He is a Docker Captain and holds ITIL Foundation Certificate in IT Service Management and Hashicorp Terraform Associate Certification. He currently works at Spacelift, and in his free time, he blogs at techblog.flaviusdinu.com, where he provides tutorials, tips, and tricks for all levels of experience based on his exposure.

YouTube

Read also

TERRAFORM 17 min read
24 Most Useful Terraform Tools to Use in 2025

TERRAFORM 19 min read
11 Popular Terraform Alternatives You Should Know in 2025

TERRAFORM 22 min read
20 Terraform Best Practices to Improve Your TF Workflow

spacelift

Get our newsletter Subscribe

Product
Documentation
Why Spacelift
How it works
Spacelift Tutorial
Pricing
Customer Case Studies
Integrations
Security

Company
About Us
Careers
Contact Sales
Partners
Media resources
Accessibility

Learn
Blog
Atlantis Alternative
Terraform Cloud Alternative
Terraform Enterprise Alternative
Spacelift for AWS
Terraform Automation
Achieve Terraform at Scale
CI/CD for Infrastructure

© 2025 Spacelift, Inc. All rights reserved